

① Lab 2

① Stack, (Push)

① $top = -1$;

1. $top == stack.length - 1$

$-1 == 4 - 1$

$\therefore$ assume 4 length of stack.

2. no stack overflow.

② $stack[++top] = 0$

$\therefore [10$

again $top == stack.length - 1$, $\neq 0 = 3 \times$, no stack overflow

$stack[++top] = 1$

$[10 \ 20$

Peek() = 20 (top element)

③ $top = 1$, $1 == 3 \times$, no stack overflow

$++top = 2$

$[10 \ 20 \ 30$

Peek() = 30

④ $top = 2$, $2 == 3 \times$.

$++top = 3$

$[10 \ 20 \ 30 \ 40]$

Peek() = 40

⑤ $top = 3$, $3 == 3$ stack overflow

$\therefore [10, 20, 30, 40]$

Pop: $[10, 20, 30, 40]$

① $top = 3$

is empty $top == stack.length - 1$

$\therefore 3 == -1 \times$

$\therefore$ no stack underflow.

$top-- = 2$

$\therefore [10, 20, 30]$

Peek() = 30

② $top = 2$

$2 == -1$ X, no stack underflow.

$top-- = 1 \therefore [10, 20]$ $peek() = 20$

③ $top = 1$

$1 == -1$ X, no stack underflow

$top-- = 0 \therefore [10]$ $peek() = 10$

④ $top = 0$

$0 == -1$ X, no stack underflow

$top-- = -1 \therefore []$

⑤ $top = -1$

$-1 == -1$ ✓, stack underflow

Queue (enqueue)

$front = -1$

$rear = -1$

① $rear == max - 1$ X $\therefore$ assume $max = 4$

$-1 == 4 - 1$ X, no queue overflow

$front == -1$

$-1 == -1$ so $front = 0$

$++rear = 0$

$\therefore [10]$

$\begin{matrix} p \\ r \end{matrix}$

② $f = 0, r = 0, 0 == 3$ X, no queue overflow

$0 == -1$ X $\therefore f = 0$

$++r = 1$

$\therefore [10, 20]$

$\begin{matrix} f \\ r \end{matrix}$

③ $f = 0, r = 1, 1 == 3$ X, no queue overflow

$1 == -1$ X $\therefore f = 0$

$++r = 2$

$\therefore [10, 20, 30]$

$\begin{matrix} f \\ r \end{matrix}$

- ④ $f=0, r=2, 2 \neq 3 \times$, no ~~stack~~^{queue} overflow
 $0 \neq -1 \times, f=0$
 $++r = 3$
 $\therefore [10, 20, 30, 40]$
- ⑤ $f=0, r=3, 3 \neq 3 \checkmark$, queue overflow
 stop $\therefore [10, 20, 30, 40]$

(dequeue:)

- ① $[10, 20, 30, 40]$ $f=0, r=3$
 f r remove (dequeue) $\rightarrow [20, 30, 40]$
 f r
- ② $\text{Queue}[front++] = f=1$
 $front > rear \rightarrow 20 > 40 \times \rightarrow [30, 40]$
 20 removed (dequeue)
 f r
- ③ $\text{queue}[front++] = f=2$
 $front > rear \rightarrow 30 > 40 \times \rightarrow [40]$
 30 dequeue
 f r
- ④ $front++ = f=3$
 $front > rear \rightarrow 40 > 40 \times \rightarrow []$
 40 dequeue
- ⑤ $front++ = f=4$
 $front > rear \rightarrow 50 > 40$, queue underflow
 $\therefore f=-1, r=-1$
 $\therefore front == -1, -1 == -1 \checkmark$ queue underflow

② CircularDeque <T> ::

Operation	Array	Front	Size
1. Initial	$[null, null, null, null, null]$	0	0
2. addlast(10)	$[10, null, null, null, null]$	0	1
3. addlast(20)	$[10, 20, null, null, null]$	0	2

0

3

1

2

Q

3

1

2

2

3

4

3

4

3

0

1

5

20

1

peekFront = 1 & peekLast = 20

In both we use sequence as:

engene (40)

engine (50)

dégénère()

engue (60)

peek()

Represent: Array = [60, 20, 30, 40, 50]

$$H_0: \rho = 1$$

count = 5

because the queue has wrapped around.

One empty slot approach:

Capacity 5, this version can contain only 4 elements. So when we reach: enqueue(50) The queue is full:
20 30 40 50

Here, one array position must remain empty
∴ later enqueue(60) will throw Queue is full.

Difference.

Feature	Count approach	One empty slot
1. Size/count field	Yes	No
2. Capacity 5 slots	5 elements	4 elements
3. Empty condition	count == 0	front == rear
4. Full condition	count == data.length	(rear + 1) % length == front.
5. Uses all array slot	Yes	No

Home challenge.

- ① Queue Using Two stacks - enqueue pushes onto stack A. Dequeue pops from stack B. If B is empty, first moves everything from A into B. Why it is this amortised $O(1)$!

Ans: Stack A → used for enqueue.
Stack B → used for dequeue.

1. Enqueue - simply push into stack A:

enqueue(10)

A = [10]

enqueue(20) - A = [10, 20]

enqueue(30) - A = [10, 20, 30]

2. Dequeue. If stack B is empty, move everything from A to B.

Page No. _____
Date / /

$A: [10, 20, 30] \rightarrow B: [30, 20, 10]$.

New pop from B:

dequeue $() \rightarrow 10$

This gives FIFO, even though stacks are LIFO.

Why is it amortised $O(1)$.

n pushes into A $\rightarrow O(n)$

n move A $\rightarrow B \rightarrow O(n)$

n pops from B $\rightarrow O(n)$

Total - $O(n) + O(n) + O(n) = O(3n) = O(n)$

for n queue operatn. - Total = $O(n)$ cost

Average cost ofth operation = $O(n)/n = O(1)$

$\therefore$ each queue operation is amortised $O(1)$.